

Лабораторная работа № 3. Работа с контроллерами и методами действия. Создание и использование фильтров

Часть I (на оценки «хорошо» и «отлично»)(2,0 балла)

1) Изучить п. «Контроллеры», «Получение данных запроса в контроллер» и «Генерация ответа из контроллеров» главы 4 «Контроллеры, действия и представления», п. «Создание HTML-элементов» главы 6 «Вспомогательные методы и работа с AJAX».

2) В среде **Visual Studio** создать новый проект **ASP.NET** для платформы **.NET Framework** (шаблон – **MVC**).

3) Добавить класс, реализующий интерфейс **ApiController**, а также обычный контроллер с методом **Index()**.

4) В методе **Execute()**, проверив, что название метода действия – «**start**», а значение переменной **id** – 0, выполнить переход на метод **Index()** обычного контроллера. Если условия не выполняются, вывести сообщение об ошибке, а также (с новой строки) полный **URL** (использовать контекстный объект **Request.Url**).

5) Для метода **Index()** создать представление, в котором, используя вспомогательные методы **Html.TextBox**, **CheckBox** и т.д., обеспечить ввод данных из предметной области по дисциплине «Базы данных» – 5-6 полей, имеющих не менее 3 различных типов, в том числе идентификатор числового типа, допускающего значение **null** (класс модели не создавать).

6) Создать **Post**-метод **Index()**, в котором получить введенные данные (частично из параметров метода действия, частично с помощью контекстного объекта **Request.Form**) и вывести их в ещё одном представлении, передав значения с помощью объекта **ViewBag**, если числовой идентификатор не пустой. Иначе выполнить перенаправление снова на метод действия **Index()**, используя метод **RedirectToAction()** (вариант 1) или **RedirectToRoute()** (вариант 2).

7) Выполнить, при необходимости, дополнительные задания, предложенные преподавателем.

Часть II (на оценку «отлично»)(1,5 балла)

1) Изучить главу 5 «Фильтры».

2) Создать пользовательский фильтр авторизации – потомка класса **AuthorizeAttribute**, в котором, используя параметр типа **HttpContextBase** метода **AuthorizeCore()**, получить значения двух переменных строки запроса, которые должны принадлежать, соответственно, двум указанным типам:

вариант 1: **string**, **int**,

вариант 2: **int**, **float**,

вариант 3: **sbyte**, **bool**,

вариант 4: **bool**, **double**,

вариант 5: **ushort**, **decimal**,

вариант 5: **float**, **string**,

вариант 6: **double**, **bool**,

вариант 7: **decimal**, **sbyte**,

вариант 8: **long**, **string**.

вариант 9: **short**, **double**.

вариант 10: **byte**, **float**.

вариант 11: **ulong**, **double**.

вариант 12: **uint**, **decimal**.

Для проверки принадлежности числовому или логическому типу применять подходящий метод класса **Convert** или метод **Parse()** соответствующего типа.

3) Применить созданный фильтр авторизации ко всем методам приложения, созданном в части I.

4) Добавить в раздел **<system.web>** файла **Web.config** следующую строку, разрешающую получение специальных ошибок: **<customErrors mode="On"></customErrors>**.

5) Выполнить обработку исключений **ArgumentOutOfRangeException** (возникает при числе переменных в строке запроса, меньшем 2) и **FormatException** (возникает при ошибке конвертации в тип, отличный от **string**) с помощью фильтров исключений:

вариант А: **ArgumentOutOfRangeException** – встроенный фильтр **HandleErrorAttribute**, **FormatException** – собственный.

вариант Б: **ArgumentOutOfRangeException** – собственный фильтр, **FormatException** – встроенный фильтр **HandleErrorAttribute**.

Применить фильтры исключений к каждому методу действия, к которому применяется фильтр авторизации.

6) Для каждого фильтра создать своё представление со сведениями об исключении, включающими, в частности, значение, которое вызвало ошибку.

7) Создать фильтр действия для **Post**-метода **Index()**. В его методе **OnActionExecuting()**, используя коллекцию **ActionParameters**, определить значение числового идентификатора – параметра метода **Index()**. Если значение отрицательно, создать экземпляр объекта **ViewResult**, установить название представления и модель (свойство **ViewData.Model**). В качестве модели использовать коллекцию **ActionParameters**. Заменить созданным объектом **ViewResult** результат метода действия.

8) Создать представление соответствующего типа. В нём, используя цикл, вывести названия переданных параметров и их значения.

9) Выполнить, при необходимости, дополнительные задания, предложенные преподавателем.